BACKEND GIT PUSH SECURITY AUDIT REPORT

PUSH REJECTED

Comprehensive Pre-Commit & Repository Exposure Assessment | EFV Platform Backend

TARGET WORKSPACE	AUDIT DATE & TIME	AUDITOR ROLE	EXECUTION POLICY
EFV-B-main (Backend)	November 04, 2026 | 11:35 IST	Lead Git Security Engineer & DevSecOps	Strict Read-Only | Secret-Masked

🛑 CRITICAL PUSH VERDICT: DO NOT PUSH TO GITHUB (READINESS: 0%)

An exhaustive read-only inspection of the backend repository discovered **catastrophic security vulnerabilities**, live administrative credentials, customer Personally Identifiable Information (PII), official tax invoices, active SQLite database records, Cloud Run production server logs, and 6.8+ GB of raw audio and document assets that are completely un-ignored by `.gitignore`. Pushing this codebase in its current state will immediately expose live production infrastructure to public takeover and severe regulatory non-compliance.

5	8	6	4	6.8 GB
P0 - CRITICAL LEAKS	P1 - HIGH VULNERABILITIES	P2 - MEDIUM EXPOSURES	P3 - LOW / HYGIENE	UNTRACKED BLOAT / MEDIA

A. Backend File Discovery & Structure Analysis

FILESYSTEM TREE

The backend contains a hybrid microservice architecture comprising a Node.js Express server (v1.6) and a Django 5.2 application (`apps/lulu`). Total uncompressed directory size is **7.0 GB** across 1,180+ files.

Component / Directory	Size / Files	Contents & Classification	Push Status
src/uploads/audios/	6.8 GB (58 files)	Heavy binary audiobooks and multi-track audio files. Violates GitHub file/repo size limits.	MUST EXCLUDE
src/uploads/ebooks/	8.0 KB (3 files)	Contains live customer Tax Invoice PDF (`INV-518743897`) and applicant resumes.	MUST EXCLUDE
src/uploads/rag/	42.1 MB (11 files)	Internal proprietary architecture PDFs (AI-Mall, UWO Platform, AMC contracts, client documents).	MUST EXCLUDE
db.sqlite3	143.3 KB	Live SQLite database with Django superuser hash, Lulu webhook logs, and print job records.	MUST EXCLUDE
logs.json	295.2 KB (100 logs)	GCP Cloud Run production logs export exposing commit SHAs, project numbers, and API traces.	MUST EXCLUDE
src/data/*.json	204 KB (38 files)	Contains `admins.json` (admin bcrypt hashes), `users.json` (customer PII), partners & tokens.	MUST EXCLUDE
Root Diagnostic JS Scripts	41 files	Ad-hoc maintenance scripts with hardcoded MongoDB Atlas URIs, Lulu secrets, and admin seeds.	MUST EXCLUDE
Root & Src .txt Dumps	16 files	Terminal output dumps (`partners_dump.txt` , `manifest_res.txt` with live courier AWBs).	MUST EXCLUDE
test*.xlsx	52 KB (3 files)	Internal test pricing spreadsheets (`test.xlsx` , `test2.xlsx` , `test_vol1.xlsx`).	MUST EXCLUDE
src/ & config/ Core Code	~1.2 MB	Legitimate application backend logic: controllers, routes, models, middleware, Django app.	SAFE (SANITIZED)

B. Files That Must Not Be Pushed (Detailed Inventory)

EXCLUSION TARGETS

1. Live MongoDB Atlas Connection Strings with Admin Plaintext Password

P0 - CRITICAL

Hardcoded live MongoDB Atlas connection URI with administrator credentials discovered in multiple root scripts:
`mongodb+srv://admin_db_user:Mf************RY@efv.adnatm4.mongodb.net/?appName=EFV`
Exposed in: `check-last-orders.js` (line 6), `check-orders-today.js` (line 6), `check-partner-orders.js` (line 6), `dump-partners-coupons.js` (line 6), `find-abha-codes.js` (line 6), `find-abha.js` (line 6).
Impact: Any actor with this URI gains unauthenticated root read/write/drop access to the entire live cluster.

2. Live Lulu Sandbox Client ID & Secret Key

P0 - CRITICAL

Hardcoded client credentials embedded directly in test script:
`process.env.LULU_SANDBOX_CLIENT_ID = '753b3d01-11e6-429a-85ac-75be197577e6'`
`process.env.LULU_SANDBOX_CLIENT_SECRET = 'GT1RBM********iw9'`
Exposed in: `scratch/test_lulu_tracking_sync.js` (lines 7–8).

3. Live Populated Database File: db.sqlite3

P0 - CRITICAL

SQLite database containing Django staff/admin superuser records (`admin` , `pbkdf2_sha256$1...`), 1 Lulu webhook log payload containing live customer order details, and 10 Lulu print jobs.
Location: `db.sqlite3` (Root level, 140 KB).

4. Customer Personally Identifiable Information (PII) & Tax Invoice

P0 - CRITICAL

A real system-generated GST Tax Invoice PDF was left in the uploads directory:
File: `src/uploads/ebooks/ebook-1774421988784-900261396-efv_invoice_69bd30f6a5cbe4ee109baf5c.pdf`
Leaked Data: Invoice `INV-518743897` , Customer `Abha` , Phone `898288****` , City `Jabalpur 482001` , GSTIN `23EFVPA0000Z1Z1` , Company Support `admin@uwo24.com` .

5. JSON Database Files with Admin Hashes & User Addresses

P0 - CRITICAL

The folder `src/data/` and root JSON files contain complete database backups:
- `src/data/admins.json` : Admin email `admin@uwo24.com` and 60-char bcrypt password hash.
- `src/data/users.json` : 10 live user accounts with bcrypt hashes, Google IDs, and saved shipping addresses.
- `src/data/partners.json` & `partners_current.json` : Partner phone numbers, session tokens, and OTPs.
- `seedAdmin.js` : Plaintext default admin password `password = 'efv@1234'` .

6. Production Cloud Logging Export: logs.json

P1 - HIGH

Contains 100 raw GCP Cloud Logging entries (UTF-16 LE). Exposes internal project numbers (`743928421487`), live Cloud Run URLs (`https://efv-743928421487.asia-south1.run.app`), API endpoints, admin product deletion traces, and previous build commit SHAs (`4f46d53fcd5f89bf28a82153bfc5c5972003acc0`).

7. Partner Database Dump: partners_dump.txt

P1 - HIGH

Plaintext terminal dump containing live partner profile: name `Devansh Tantway` , email `devanshtantwaynir@gmail.com` , phone `0769404****` , bcrypt password hash `$2a$10$9Abrm1tAJ1IOM...` , auth token `69553fc801aefc412eda42d4bf30b9c0` , and partner token `DEVANSH-EFV-7401` .

8. Proprietary Enterprise IP & Contract Documents in src/uploads/rag/

P1 - HIGH

Contains 42 MB of un-redacted architectural and commercial PDFs:

- `AI-Mall_Autonomous_Infrastructure 2.pdf` (19.6 MB)
- `UWO_Unified_AI_Platform.pdf` (9.4 MB)
- `yug amc dfoc.pdf` (5.7 MB) - Commercial Annual Maintenance Contract document.
- Candidate CVs: `src/uploads/ebooks/devansh_resume.pdf` .

9. Hardcoded Weak Fallback Secret ("secret123") in Authentication Handlers

P1 - HIGH

Widespread use of insecure fallback: `jwt.verify(token, process.env.JWT_SECRET || 'secret123')` .
Present in: `src/middleware/auth.js` , `src/middleware/adminAuth.js` , `src/middleware/partnerAuth.js` , `src/routes/auth.js` , `src/routes/partnerPortal.js` .
Also in `test_upload.js` : Token forged with `'efv_secret_key_2026'` .
In `config/settings.py` : Insecure Django Secret Key `django-insecure-*%0un3kd+hk...` with `DEBUG = True` .

10. Live Courier AWBs & Nimbus Tracking Logs

P2 - MEDIUM

Airway Bill tracking numbers and NimbusPost manifest generation responses exposed in:
`src/manifest_res.txt` & `src/status_res.txt` (AWBs: `77712193576` , `77712192913` , `14227850765371`).

C & D. Git Tracking & History Analysis

VULNERABILITY STATE

Inspection Command	Observed Output / Status	Security Auditor Analysis
`git status --short`	`fatal: not a git repository`	Directory was unzipped from an archive without `.git` metadata. Preparing for initial repo setup.
`git ls-files`	0 files tracked locally	No local tracking currently active.
`git diff --cached`	0 files staged	No staged files in current working tree.
`git check-ignore -v` (Simulated)	**0 rules match sensitive files**	`db.sqlite3` , `logs.json` , `src/data/*.json` , `*.txt` , `src/uploads/` are **completely un-ignored**!
`git log --all`	No commits in local tree	External commit SHA discovered in logs: `4f46d53fcd5f89bf28a82153bfc5c5972003acc0` .

⚠️ Catastrophic Initial Commit Risk Simulation

STAGING SIMULATION

If a developer executes `git init && git add .` right now:
72+ sensitive files, 6.8+ GB of media, and 5 P0 credential leaks will be instantly staged and committed! The current `.gitignore` only excludes `node_modules/` , `.env` , and `output.txt` . It fails to protect any database files, JSON data, uploads, or logs.

E. Git Ignore Review & Discrepancy Analysis

CONFIGURATION DRIFT

An audit of `.gitignore` vs. `.gcloudignore` and `.dockerignore` revealed alarming discrepancies. The developers configured exclusions for Google Cloud deployment but completely omitted them from Git!

File / Pattern	In .gitignore?	In .gcloudignore?	Required Action
`*.sqlite3 / db.sqlite3`	❌ **NO (Exposed!)**	❌ NO	ADD IMMEDIATELY

File / Pattern	In .gitignore?	In .gcloudignore?	Required Action
`src/uploads/`	❌ **NO (6.8 GB Exposed!)**	✅ YES	ADD IMMEDIATELY
`public/uploads/`	❌ **NO (Exposed!)**	❌ NO	ADD IMMEDIATELY
`logs.json / *.log`	❌ **NO (Exposed!)**	✅ YES	ADD IMMEDIATELY
`*.txt (Output Dumps)`	❌ **NO (Exposed!)**	✅ YES	ADD IMMEDIATELY
`src/data/*.json (DB State)`	❌ **NO (Exposed!)**	❌ NO	ADD IMMEDIATELY
`*.xlsx (Pricing Spreadsheets)`	❌ **NO (Exposed!)**	❌ NO	ADD IMMEDIATELY
`__pycache__/ & *.pyc`	❌ **NO (Exposed!)**	❌ NO	ADD IMMEDIATELY
`scratch/ & src/scratch/`	❌ **NO (Exposed!)**	❌ NO	ADD IMMEDIATELY

F. Final Classification & Action Plan

ACTION REQUIRED

1. Backend Files Safe to Push (Legitimate Source Code)

- `src/server.js` (after removing debug URLs)
- `src/config/db.js`, `src/config/vertex.js` (standard client initializations)
- `src/routes/`: Core business routes (`auth.js`, `orders.js`, `products.js`, etc.) — *must remove fallback 'secret123'*
- `src/models/`: Mongoose schemas and data modeling
- `src/middleware/`: Token validation and role check middleware
- `src/services/`: Business service handlers (`nimbusPostService.js`, `notificationWorker.js`, etc.)
- `src/utils/`: Helper modules (`cashfree.js`, `lulu.js`, `emailService.js`)
- `apps/lulu/`: Django REST framework integration module
- `config/`: Django project configuration (`urls.py`, `wsgi.py`, `asgi.py`)
- `package.json` & `package-lock.json` (legitimate dependency manifests)
- `Dockerfile`, `manage.py`

2. Backend Files That MUST NOT Be Pushed

- **All Database Files:** `db.sqlite3`
- **All State & Mock Database JSON Files:** `src/data/admins.json`, `src/data/users.json`, `src/data/partners.json`, `src/data/demo_users.json`, `src/data/system_settings.json`, `coupons_current.json`, `partners_current.json`, `legacy_ids.json`, `test_prod.json`, `lib_error_logs.json`
- **All Uploaded Media & Customer Documents:** `src/uploads/` (all audios, customer invoice PDF, RAG business architecture PDFs, resumes), `public/uploads/`
- **All Production Logs & Dumps:** `logs.json`, `admin_lib_dump.txt`, `admin_lib_list.txt`, `audio_out.txt`, `bucket_out.txt`, `bucket_out2.txt`, `check_output.txt`, `check_result.txt`, `digital_products.txt`, `legacy_ids.txt`, `origin_code.txt`, `origin_full.txt`, `out.txt`, `partners_dump.txt`, `products_log.txt`, `server_output.txt`, `server_output_utf8.txt`, `src/manifest_res.txt`, `src/status_res.txt`
- **All Ad-hoc Diagnostic Scripts:** `check-last-orders.js`, `check-orders-today.js`, `check-partner-orders.js`, `dump-partners-coupons.js`, `find-abha-codes.js`, `find-abha.js`, `seedAdmin.js`, `test_upload.js`, `scratch/`, `src/scratch/`
- **Excel Spreadsheets & Binary Caches:** `test.xlsx`, `test2.xlsx`, `test_vol1.xlsx`, `__pycache__/`, `*.pyc`

3. Required Remediation Roadmap (Step-by-Step)

1. **Immediate Credential Revocation (Priority 0):**
   Rotate the exposed MongoDB Atlas user password for `admin_db_user` immediately via the MongoDB Atlas web console. Rotate the Lulu API client secret and Cashfree API keys.
2. **Enforce Production Environment Variables & Remove Fallbacks:**
   Remove the dangerous fallback `'secret123'` from all authentication and JWT verify functions. Ensure application throws a fatal startup exception if `process.env.JWT_SECRET` is not set. Change Django `DEBUG = False` in `config/settings.py` and mandate an external `SECRET_KEY`.
3. **Sanitize Root Directory:**
   Move all 41 exploratory and test scripts into an un-tracked developer scratch directory or delete them entirely.
4. **Adopt Enterprise Cloud Storage for User Uploads:**
   Ensure all audiobooks, PDFs, and cover images reside exclusively on Google Cloud Storage (GCS) or S3. Never store user media on the local filesystem of application containers.
5. **Implement Production .gitignore (Provided Below):**
   Replace the existing `.gitignore` with the comprehensive ruleset below prior to running `git init`.

4. Recommended Production .gitignore Rules

```
# ============================================================
# EFV Platform Backend - Security Hardened .gitignore
# ============================================================

# Environment files
.env
.env.*
!.env.example

# Local SQLite & Databases
*.sqlite3
*.db
db.sqlite3
```

```
# JSON Data Stores & State Backups
src/data/*.json
!src/data/categories.json
coupons_current.json
partners_current.json
legacy_ids.json
test_prod.json
logs.json
lib_error_logs.json

# Local Uploads & Blobs (Stored in Cloud Buckets)
src/uploads/
public/uploads/
*.mp4
*.mp3
*.wav

# Log Files & Terminal Dumps
*.log
*.txt
!package*.json
logs/
npm-debug.log*

# Office & Test Spreadsheets
*.xlsx
*.xls
*.csv

# Python Bytecode & Virtual Envs
__pycache__/
*.py[cod]
*$py.class
venv/
.env/

# Diagnostic Scripts & Scratch Dirs
scratch/
src/scratch/
check-*.js
check_*.js
find-*.js
genHash*.js
seedAdmin.js
test_*.js
test-*.js
migrate*.js
clean_products.js
update_mongo_products.js

# Dependencies & Build Artefacts
node_modules/
dist/
build/
.cache/
```

FINAL AUDIT DECISION DO NOT PUSH

The backend is **UNSAFE FOR GIT PUSH**. The development team must perform the step-by-step remediation outlined above, verify that the updated `.gitignore` suppresses all 72+ non-code files, and re-run an automated pre-push security audit before initializing Git.